Hibabejelentő és karbantartás-kezelő rendszer

Szerzők:

Farkas Levente, Horváth Dániel, Kantó József

Témavezető:

Dr. Tarczali Tünde Tímea, adjunktus

2026

Kivonat

A projekt célja egy olyan asztali hibabejelentő és karbantartás-kezelő rendszer megtervezése és fokozatos megvalósítása, amely támogatja egy szervezet műszaki, jelen projektben elsősorban informatikai hibáinak bejelentését és kezelését. A rendszerben a felhasználók bejelentéseket hozhatnak létre, követhetik azok állapotát, a karbantartók pedig a hozzájuk rendelt feladatokat kezelhetik.

A rendszer tervezése során meghatároztuk a fő felhasználói szerepköröket, a bejelentések kezelésének tervezett folyamatát, valamint az alkalmazás alapvető architektúráját. A tervezett megoldás .NET MAUI asztali kliensből, .NET Web API-ból és a szerveroldalon kezelt SQLite-adatbázisból áll. A kliens HTTP-kéréseken keresztül kommunikál a szerverrel, az üzleti logikát és az adatbázis-hozzáférést pedig a Web API kezeli.

A jelenlegi prototípus az API és az adatbázis kapcsolatának ellenőrzésére alkalmas. A felhasználói azonosítás, a hibabejelentések teljes kezelése, a karbantartói munkafolyamat és az adminisztráció a további fejlesztés feladata. A dokumentáció különbséget tesz a már elkészült prototípus és a tervezett funkciók között.

Kulcsszavak: hibabejelentés, ticketkezelés, .NET MAUI, Web API, SQLite

Tartalomjegyzék

Kivonat

1. Bevezetés **1**

1.1. A projekt célja 1

1.2. A projekt hatóköre 2

1.3. A projekt tervezése 2

2. Követelmények és szerepkörök **3**

2.1. Felhasználói szerepkörök 3

2.1.1. Felhasználó és hibabejelentő 3

2.1.2. Karbantartó 3

2.1.3. Rendszeradminisztrátor 3

2.2. Regisztráció és felhasználókezelés 4

2.3. Bejelentések adatai 4

2.4. Állapotok és munkafolyamat 5

2.5. Bejegyzések és értesítések 6

2.6. Nem funkcionális követelmények 6

3. Használati esetek **8**

3.1. A bejelentő használati esetei 8

3.2. A karbantartó használati esetei 8

3.3. Az adminisztrátor használati esetei 9

4. Választott technológiák **10**

4.1. Frontend 10

4.2. Backend 10

5. Szoftver architektúra **12**

5.1. Rendszer felépítése 12

5.2. Adatbázis kialakítása 12

5.2.1. Felhasználók és karbantartói csoportok 12

5.2.2. Bejelentések és kategóriák 13

5.2.3. Prioritás és határidő 14
5.2.4. Állapotok és állapotváltozási előzmények 15
5.2.5. Bejegyzések, újranyitások és értesítések 15
5.2.6. A fő táblák és kapcsolataik összefoglalása 15
5.3. Asztali alkalmazás 16
5.3.1. Hibabejelentő GUI 16
5.3.2. IT Support GUI 16
5.3.3. Rendszeradminisztrátor GUI 17

6. Kommunikációs rendszer kialakítása (API) 18
6.1. Az API szerepe 18
6.2. Tervezett végpontok 18
6.3. Kérések és válaszok 19
6.4. Hitelesítés és jogosultság-ellenőrzés 20
6.5. A bejelentés életciklusa 20
6.6. Értesítések kezelése 21

7. Megvalósítás menete és tesztelés 22
7.1. A jelenlegi prototípus állapota 22
7.2. Javasolt fejlesztési sorrend 22
7.3. Tesztelési terv 22
7.4. Ismert korlátok és továbbfejlesztési lehetőségek 23

8. Összegzés 25

Irodalomjegyzék 26

1 Bevezetés

Egy szervezet mindennapi működését különböző műszaki problémák nehezíthetik. Ezek között lehetnek épülethez, eszközökhöz, hálózathoz vagy informatikai szolgáltatásokhoz kapcsolódó hibák. Ha a problémák bejelentése informális csatornákon, például szóban vagy e-mailben történik, nehezebb nyomon követni, hogy ki foglalkozik velük, milyen állapotban vannak, és milyen intézkedés történt.

A projekt célja egy olyan alkalmazás megtervezése és megvalósítása, amely egységes felületet biztosít a hibák bejelentésére, nyilvántartására és kezelésére. A bejelentő rögzítheti a probléma lényeges adatait, később megtekintheti saját bejelentéseinek állapotát, a karbantartást végző munkatárs pedig kezelheti a hozzá rendelt feladatokat.

A kiadott projektfeladat általános műszaki hibabejelentő rendszert ír le. A projekt jelenlegi terve ezt a feladatot egy szervezet belső, elsősorban informatikai infrastruktúrájához kapcsolódó hibakezelési folyamatára szűkíti. A dokumentációban ezt a szűkített projektkört használjuk.

1.1 A projekt célja

A rendszer célja, hogy a hibabejelentések kezelése átlátható, visszakereshető és szerepkörökhöz kötött legyen. A bejelentéseknek egyedi azonosítójuk van, és az életciklusuk során rögzülnek a hozzájuk tartozó állapotváltozások és kommunikációs bejegyzések.

A rendszer tervezett fő funkciói:

- műszaki hibák bejelentése;
- egyedi azonosító rendelése a bejelentésekhez;
- a bejelentések kategorizálása és priorizálása;
- bejelentések karbantartói csoporthoz rendelése;
- a csoporttagok számára a csoporthoz rendelt feladatok megtekintése;
- a feladatok átvétele és kezelése;
- a bejelentés állapotának követése;
- kommunikációs bejegyzések és javítási információk rögzítése;
- alkalmazáson belüli értesítések megjelenítése;

– felhasználók, csoportok és kategóriák adminisztrációja.

1.2 A projekt hatóköre

A projekt első változata a bejelentés létrehozására, nyomon követésére és kezelésére összpontosít. A rendszer felhasználói bejelentői, karbantartói és adminisztrátori feladatokat látnak el. A karbantartó nem feltétlenül külön felhasználói típus: a karbantartói feladatokat olyan felhasználó végzi, aki egy karbantartói csoport tagja.

A jelenlegi tervben a keresés az egyedi ticketazonosító alapján történik. A kategória, állapot, prioritás vagy dátum szerinti részletes szűrés későbbi bővítésként kezelhető. E-mailes értesítés nem része az első változat követelményeinek; a rendszer alkalmazáson belüli értesítéseket használ.

1.3 A projekt tervezése

A munka a kiadott specifikáció elemzésével kezdődött. A csapat meghatározta a fő funkciókat, a tervezett felhasználói szerepköröket, valamint a bejelentések kezelésének alapvető folyamatát. A tervezéshez használati eseteket és adatbázistervet készítettünk.

A rendszer központi eleme a hibabejelentés, más néven ticket. A felhasználó létrehozza a bejelentést, az arra jogosult személy szükség esetén kategorizálja és priorizálja, majd karbantartói csoporthoz rendeli. A csoport tagjai megtekintik a feladatot, és egyikük átveheti annak kezelését. A karbantartó bejegyzéseket és javítási információkat rögzíthet, majd lezárhatja a bejelentést. A bejelentő követheti a folyamatot, válaszolhat a karbantartónak, és indoklással újranyithatja a lezárt bejelentést.

2 Követelmények és szerepkörök

2.1 Felhasználói szerepkörök

A rendszer három fő szerepkört különít el: bejelentő, karbantartó és adminisztrátor. A jogosultságok meghatározzák, hogy az adott felhasználó mely bejelentéseket láthatja, illetve milyen műveleteket hajthat végre.

2.1.1 Felhasználó és hibabejelentő

A hibabejelentő a rendszer alapvető felhasználója. Regisztráció és adminisztrátori jóváhagyás után bejelentkezhet az alkalmazásba. Új hibabejelentést hozhat létre, megtekintheti saját bejelentéseit, és egyedi azonosító alapján kereshet közöttük.

A bejelentő kommunikációs bejegyzést írhat a saját ticketjéhez. Lezárt bejelentését indoklással újranyithatja. A felhasználó megtekintheti a számára létrehozott alkalmazáson belüli értesítéseket is.

2.1.2 Karbantartó

A karbantartó olyan felhasználó, aki egy karbantartói csoport tagja. A csoporthoz rendelt ticketeket megtekintheti, és a csoport tagjaként magára veheti valamelyik feladatot. A karbantartó kommunikációs bejegyzést írhat, rögzítheti az elvégzett munkával kapcsolatos információkat, frissítheti a bejelentés állapotát, és lezárhatja a feladatot.

A karbantartó a bejelentői funkciókat is használhatja, tehát saját maga is létrehozhat hibabejelentést.

2.1.3 Rendszeradminisztrátor

Az adminisztrátor felelős a rendszer alapadatainak és felhasználóinak kezeléséért. Jóváhagyhatja a regisztrációkat, inaktiválhat felhasználókat, csoporthoz rendelheti őket, valamint kezelheti a karbantartói csoportokat és kategóriákat.

Az adminisztrátor a bejelentéseket áttekintheti, karbantartói csoporthoz rendelheti, illetve módosíthatja azok prioritását. Az adminisztrátori fiók külön jogosultsági kategória; az adminisztrátori felhasználó nem kerül automatikusan karbantartói csoportba.

A prioritásértékek és a hozzájuk tartozó határidő-szabályok kódban definiált beállítások; ezeket

az adminisztrátor nem módosíthatja az alkalmazásból.

2.2 Regisztráció és felhasználókezelés

A regisztrációs folyamat tervezett lépései:

1. A felhasználó regisztrációt kezdeményez.
2. A regisztráció adminisztrátori jóváhagyásra vár.
3. Az adminisztrátor áttekinti a regisztrációt.
4. Jóváhagyás után a felhasználó aktívvá válik.
5. Az aktív felhasználó bejelentkezhet az alkalmazásba.

A regisztráció és a bejelentkezés külön funkció. A regisztráció felhasználói fiókot kezdeményez, a bejelentkezés pedig egy már jóváhagyott fiók használatát teszi lehetővé.

A jelenlegi követelmények szerint a felhasználó manuális létrehozása, általános adatainak módosítása és törlése nem része az adminisztrátori funkcióknak. A felhasználó inaktiválható, jóváhagyható, illetve karbantartói csoporthoz rendelhető. Egy felhasználó legfeljebb egy karbantartói csoport tagja lehet.

2.3 Bejelentések adatai

A bejelentés az alábbi adatokat tartalmazza:

- egyedi ticketazonosító;
- cím és leírás;
- a probléma jellege;
- kötelező helyszín;
- opcionális eszközazonosító;
- kategória;
- prioritás;
- határidő;
- aktuális állapot;

– bejelentő;

– hozzárendelt karbantartói csoport;

– aktuális kezelő, ha egy karbantartó átvette a feladatot;

– a bejelentéshez tartozó bejegyzések;

– létrehozás, módosítás és lezárás időpontja.

A bejelentés helyszínének megadása kötelező, mert ez segíti a hiba beazonosítását és a szükséges intézkedés megtervezését. Az érintett eszköz azonosítója opcionális, mivel nem minden probléma köthető egy konkrét eszközhöz.

A prioritást a rendszer négy, kódban definiált érték egyikeként kezeli: Alacsony, Normál, Magas vagy Sürgős. Ezeket az adminisztrátor nem módosíthatja az alkalmazásban. A fejlesztők számára a prioritások és a hozzájuk rendelt határidők egy központi kódbeli beállításban módosíthatók.

A határidő munkanapokban számítódik. Az első változatban munkanapnak a hétfőtől péntekig tartó napok számítanak; a munkaszüneti napok külön naptári kezelése nem része a rendszernek. A tervezett alapértékek: Sürgős 1, Magas 3, Normál 5, Alacsony 10 munkanap. A határidő a bejelentés létrehozásakor, a kiválasztott prioritás alapján keletkezik.

A kategóriák adminisztrátori kezelés alatt állnak. A kategória azt jelöli, hogy a probléma melyik kezelési csoportba tartozik, míg a probléma jellege a bejelentő által megadott technikai típust írja le.

2.4 Állapotok és munkafolyamat

A jelenlegi tervben a bejelentés lehetséges állapotai:

– Nyitva;

– Folyamatban;

– Válaszra vár;

– Újranyitott;

– Lezárt – elvégezve;

– Lezárt – visszavonva;

– Lezárt – elutasítva;

– Lezárt – megkerülő megoldással.

Az alapfolyamat tervezett menete:

$$Nyitva \rightarrow Folyamatban \rightarrow Válaszra\ vár \rightarrow Folyamatban \rightarrow Lezárt\ állapot$$

A karbantartó a bejelentés kezelése során bejegyzést írhat a tickethez. Ha a bejegyzés a bejelentőtől választ vagy további információt kér, a bejelentés Válaszra vár állapotba kerül, és a bejelentő értesítést kap. A bejelentő válasza után a bejelentés automatikusan Folyamatban állapotú lesz.

A lezárást a karbantartó végzi. A lezárás módját a bejelentővel folytatott kommunikáció és a probléma megoldásának eredménye alapján választja ki. A lezárás módja lehet elvégzett munka, visszavonás, elutasítás vagy megkerülő megoldás.

A bejelentő bármely lezárt állapotból újranyithatja saját bejelentését, de ehhez indoklást kell megadnia. Az újranyitott bejelentés a felhasználó első új bejegyzésekor Folyamatban állapotba kerül.

2.5 Bejegyzések és értesítések

Egy bejelentéshez több bejegyzés tartozhat. A bejegyzések a bejelentő, a karbantartó és az adminisztrátor közötti kommunikáció előzményeit rögzítik. A karbantartó az elvégzett munka leírását is ezekben a bejegyzésekben rögzíti; külön javítási tábla nem szükséges.

A tervezett alkalmazáson belüli értesítések címzettjei:

– a bejelentő: állapotváltozás, új bejegyzés vagy újranyitás esetén;

– a karbantartó: csoporthoz rendelt ticket vagy felhasználói válasz esetén;

– az adminisztrátor: új regisztráció esetén.

A rendszer nem törli a bejelentéseket. A lezárás állapotváltozással történik, hogy a bejelentéshez tartozó előzmények megmaradjanak.

2.6 Nem funkcionális követelmények

A rendszernek a felhasználói műveleteket átláthatóan kell kezelnie. A kliens jelezze a folyamatban lévő műveleteket, a sikeres eredményt és a hibát is. A hibaüzenetek legyenek a

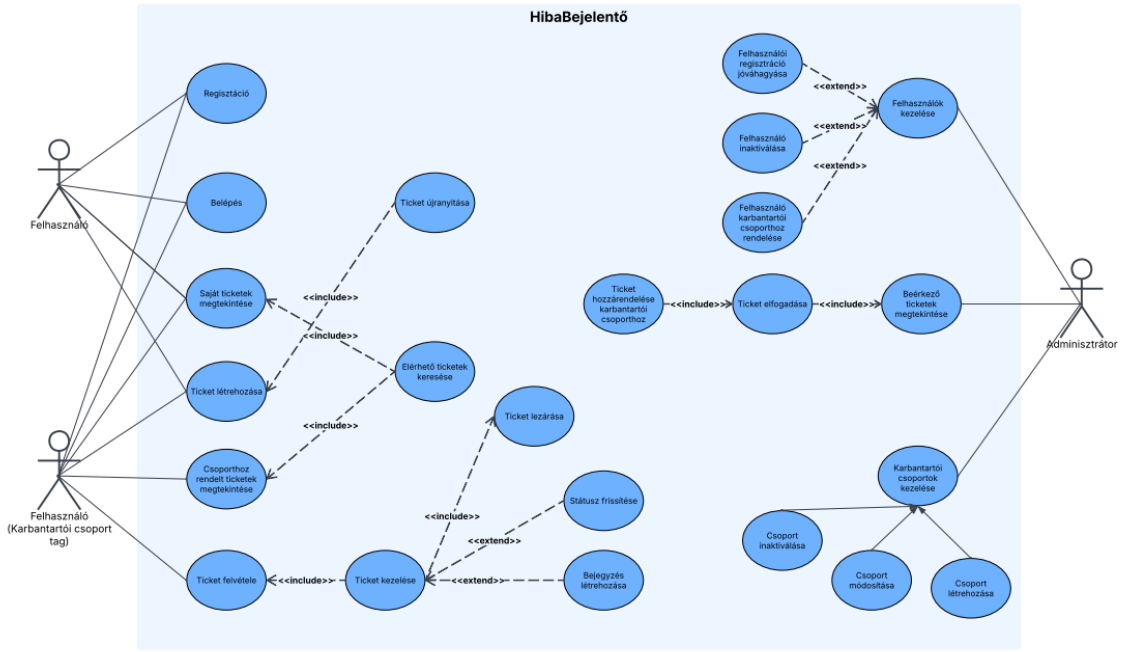
felhasználó számára érthetők, és ne tartalmazzanak belső technikai adatokat.

A szerveroldali ellenőrzésnek minden védett műveletnél érvényesülnie kell. Az adatbázis közvetlenül nem érhető el a kliensből. A jelszavakat nem szabad olvasható formában tárolni.

A rendszernek meg kell őriznie a bejelentésekhez tartozó kommunikációs előzményeket és állapotváltozásokat. A bejelentések törlése helyett lezárás vagy felhasználó-inaktiválás használható, így a korábbi adatok visszakereshetők maradnak.

3 Használati esetek

A használati esetek azt mutatják be, hogy az egyes felhasználói szerepkörök milyen funkciókat érhetnek el a rendszerben. A projekt használatieset-diagramja a bejelentő, a karbantartói csoport tagja és az adminisztrátor fő műveleteit foglalja össze.



1. ábra: A hibabejelentő rendszer használatieset-diagramja

3.1 A bejelentő használati esetei

A bejelentő regisztrációval kezdeményezheti a rendszer használatát. A regisztráció jóváhagyását követően bejelentkezhet, hibabejelentést hozhat létre, és áttekintheti a saját bejelentéseit. A bejelentés egyedi azonosítója alapján visszakeresheti az adott ticketet.

A bejelentő a tickethez kapcsolódó bejegyzésben válaszolhat a karbantartónak. Ha a bejelentés lezárult, de a probléma továbbra is fennáll, indoklással újranyithatja. A számára létrehozott alkalmazáson belüli értesítéseket szintén megtekintheti.

3.2 A karbantartó használati esetei

A karbantartó a saját bejelentésein túl láthatja a karbantartói csoportjához rendelt ticketeket. A csoporthoz rendelt bejelentést magára veheti, ezzel jelezve, hogy ő végzi a kezelését.

A ticket kezelése során bejegyzést írhat, a bejelentővel folytatott kommunikáció alapján állapotváltozást kezdeményezhet, és rögzítheti az elvégzett munka leírását. A bejelentés lezárásakor megadja a lezárás eredményét, például azt, hogy a hiba elhárult, a bejelentést visszavonták vagy elutasították, illetve megkerülő megoldást alkalmaztak.

3.3 Az adminisztrátor használati esetei

Az adminisztrátor a regisztrációkat jóváhagyhatja, felhasználókat inaktiválhat és karbantartói csoporthoz rendelhet. Kezeli a csoportokat és a hibakategóriákat, valamint bejelentéseket rendelhet karbantartói csoporthoz és módosíthatja azok prioritását.

A prioritásértékek és a határidő-szabályok nem az adminisztrátori felületen módosíthatók. Ezek a rendszer kódbeli beállításai közé tartoznak.

4 Választott technológiák

A rendszer tervezett felépítése egy asztali kliensből, egy HTTP-alapú alkalmazásprogramozási felületből (API) és egy szerveroldali adatbázisból áll. A kliens feladata a felhasználói felület megjelenítése és a felhasználói műveletek kezdeményezése. Az üzleti szabályok ellenőrzése és az adatok kezelése a Web API feladatkörébe tartozik. Az adatbázist a kliens nem éri el közvetlenül.

A jelenlegi prototípusban már létrejött a .NET MAUI kliensprojekt és a .NET Web API projekt. A kliens jelenleg az API, illetve az adatbázis elérhetőségét ellenőrzi. A hibabejelentések teljes kezelése és a jogosultsági rendszer még tervezési, illetve fejlesztési feladat.

4.1 Frontend

A kliensalkalmazás .NET MAUI keretrendszerrel, C# nyelven készül. A MAUI lehetőséget ad arra, hogy a felhasználói felület és az alkalmazás logikájának jelentős része közös projektben készüljön el. A projekt jelenlegi célplatformja a Windows asztali környezet.

A kliens a szerverrel HTTP-kéréseken keresztül kommunikál. A jelenlegi prototípusban egy szolgáltatásosztály végzi az API elérését, és az alkalmazás egy állapotellenőrző végpont válaszát jeleníti meg. A tervezett funkciók megvalósításakor hasonló módon történhet majd a bejelentések létrehozása, listázása, részleteinek megtekintése és módosítása is.

A kliens nem tárolja és nem kezeli közvetlenül az SQLite-adatbázist. Ez a kialakítás elkülöníti a felhasználói felületet az adatkezeléstől, és lehetővé teszi, hogy az üzleti szabályokat a szerver ellenőrizze.

4.2 Backend

A szerveroldali alkalmazás ASP.NET Core Web API formájában készül C# nyelven. A projekt .NET 10 célkeretrendszert használ. A Web API fogadja a kliens kéréseit, ellenőrzi azok érvényességét, végrehajtja az üzleti szabályokat, majd választ küld a kliensnek.

A jelenlegi prototípus tartalmaz egy állapotellenőrző végpontot. Ez megpróbál kapcsolatot létesíteni az adatbázissal, majd visszajelzi a szolgáltatás és az adatbázis elérhetőségét. A fejlesztői környezetben az API OpenAPI-leírást és Swagger felületet is biztosít a végpontok áttekintéséhez.

A projekt jelenlegi adat-hozzáférési rétege az Entity Framework Core SQLite-szolgáltatóját használja. A további fejlesztés során a bejelentkezést, a jogosultság-ellenőrzést, a bejelentések kezelését és az értesítések létrehozását is a szerveroldali rétegben kell megvalósítani.

5 Szoftver architektúra

5.1 Rendszer felépítése

A rendszer három fő részből áll:

- **MAUI kliens:** megjeleníti a felhasználói felületet, összegyűjti a felhasználó által megadott adatokat, és kéréseket küld az API-nak.
- **Web API:** fogadja és feldolgozza a kliens kéréseit, ellenőrzi az üzleti szabályokat, kezeli a jogosultságokat, valamint elvégzi az adatbázis-műveleteket.
- **SQLite adatbázis:** a szerveroldali alkalmazás által kezelt adatokat tárolja, például a felhasználókat, csoportokat, bejelentéseket és bejegyzéseket.

A fő kommunikációs útvonal:

`.NET MAUI kliens` → `HTTP/REST API` → `.NET Web API` → `SQLite adatbázis`

A kliens a Web API-n keresztül kezdeményez adatlekérést vagy adatváltoztatást. A szerver ellenőrzi, hogy a kérés megfelel-e a rendszer szabályainak, majd szükség esetén módosítja az adatbázist. A kliens a szerver válasza alapján frissíti a megjelenített adatokat.

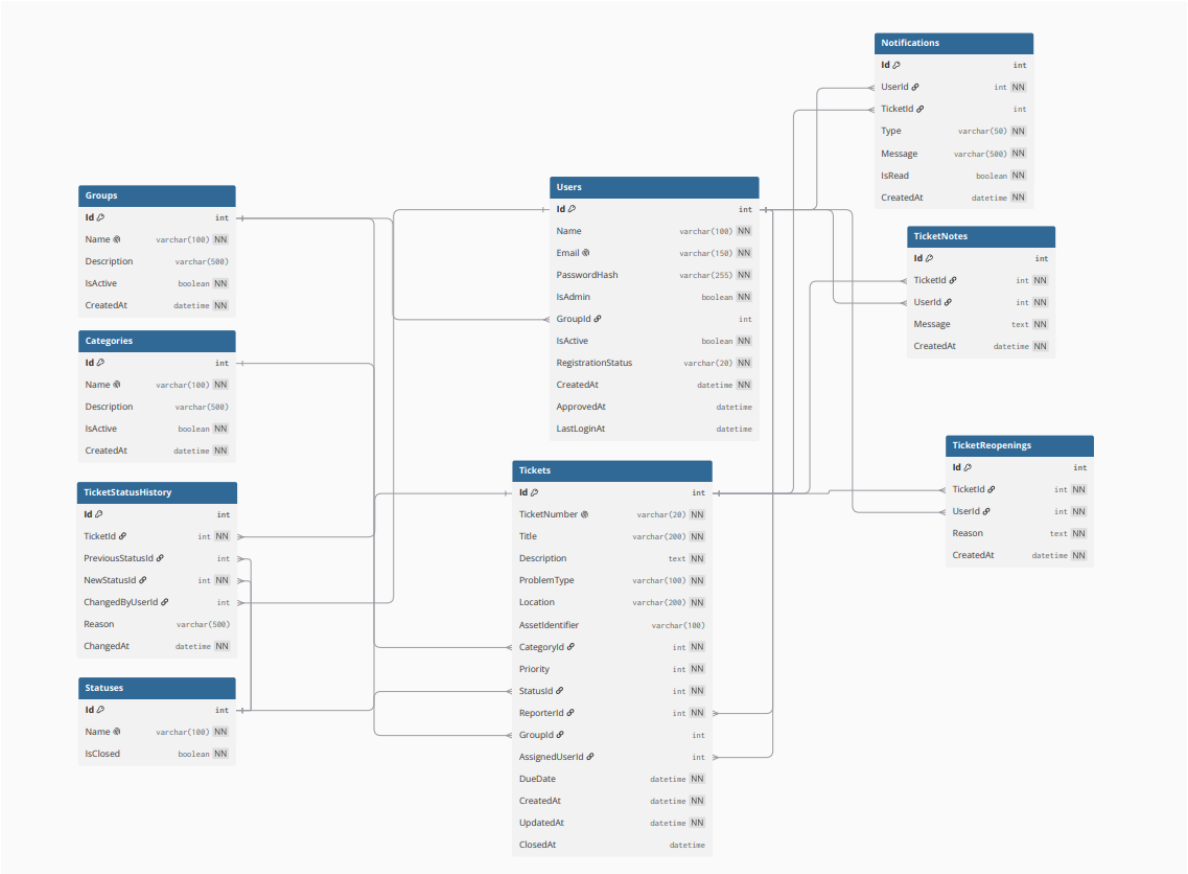
Ez az architektúra a felhasználói felületet, az üzleti logikát és az adattárolást különíti el. Ennek eredményeként az adatbázis közvetlenül nem érhető el a kliensből, az adatokat érintő döntések pedig egy helyen, a szerveren kezelhetők.

5.2 Adatbázis kialakítása

A rendszer adatbázisa a Web API által kezelt SQLite-adatbázis. Az adatmodell relációs felépítésű: a bejelentések a felhasználókhoz, csoportokhoz, kategóriákhoz és állapotokhoz kapcsolódnak. A kommunikációs bejegyzéseket, az újranyitási indoklásokat és az állapotváltozások előzményeit külön táblák tárolják.

5.2.1 Felhasználók és karbantartói csoportok

A `Users` tábla tárolja a felhasználó nevét, e-mail-címét, jelszóhash-ét, aktív állapotát és regisztrációs állapotát. Az e-mail-cím egyedi. Az adminisztrátori jogosultságot az `IsAdmin` mező jelzi.



2. ábra: A hibabejelentő rendszer tervezett adatmodellje

A karbantartói csoporttagságot a felhasználó `GroupId` mezője rögzíti. Egy csoporthoz több felhasználó tartozhat, egy felhasználó viszont legfeljebb egy csoport tagja lehet. A `Groups` tábla a csoport nevét, leírását és aktív állapotát tárolja. A csoportokat az adminisztrátor kezeli.

A felhasználók törlése helyett az inaktiválás használható. Ez megőrzi a korábbi bejelentésekhez és bejegyzésekhez fűződő kapcsolatokat.

5.2.2 Bejelentések és kategóriák

A `Tickets` tábla tartalmazza a bejelentés egyedi azonosítóját, címét, leírását, a hiba jellegét, kötelező helyszínét és opcionális eszközazonosítóját. Emellett rögzíti a bejelentőt, a kategóriát, a prioritást, az aktuális állapotot, a hozzárendelt csoportot és az aktuális kezelőt.

A `TicketNumber` mező egyedi, felhasználók számára megjeleníthető azonosítót tárol, például `INC123123` formában. A `ReporterId` jelöli a bejelentőt. A `GroupId` az illetékes karbantartói csoportot, az `AssignedUserId` pedig az adott ticketet magára vevő karbantartót jelöli.

A `Location` mező kötelező, mert a hiba helyének ismerete szükséges lehet a probléma kivizsgálásához. Az `AssetIdentifier` opcionális, mivel nem minden hiba köthető egy konkrét

eszközhöz.

A `Categories` tábla az adminisztrátor által kezelt hibakategóriákat tartalmazza. Egy kategóriához több bejelentés is tartozhat. A már használt kategóriát célszerű inaktiválni, nem törölni, hogy a korábbi bejelentések hivatkozásai megmaradjanak.

5.2.3 Prioritás és határidő

A prioritás négy rögzített értéke: Alacsony, Normál, Magas és Sürgős. Ezek C# enumként szerepelnek a programban, az adatbázis pedig a kiválasztott érték egész számú azonosítóját tárolja. A prioritásoknak nincs külön adatbázistáblájuk, és az adminisztrátor nem módosíthatja őket az alkalmazásból.

A prioritáshoz tartozó határidő-beállításokat a program egy központi helyen tartalmazza. A tervezett kezdőértékek:

- Sürgős: 1 munkanap;
- Magas: 3 munkanap;
- Normál: 5 munkanap;
- Alacsony: 10 munkanap.

A határidő számításakor a hétfőtől péntekig tartó napok számítanak munkanapnak. Az első változat nem használ külön munkaszüneti-naptárat. A ticket létrehozásakor a rendszer a prioritás alapján kiszámítja és elmenti a `DueDate` értékét.

A kódbeli prioritásértékek szemléltető példája:

```
public enum TicketPriority
{
    Alacsony = 1,
    Normal = 2,
    Magas = 3,
    Surgos = 4
}
```

A megjelenített feliratok magyar ékezetes formában jelennek meg. A `Surgos` azonosító kódbeli név; a felhasználói felületen „Sürgős” felirat tartozik hozzá.

5.2.4 Állapotok és állapotváltozási előzmények

A `Statuses` tábla tartalmazza a bejelentések lehetséges állapotait és azt, hogy az állapot lezártnak számít-e. Egy állapothoz több ticket tartozhat.

A `Tickets.StatusId` mező az aktuális állapotot jelöli. Ez önmagában nem őrzi meg, hogy korábban milyen állapotokon ment át a bejelentés. Ezért a `TicketStatusHistory` tábla minden állapotváltásról külön rekordot tárol.

Az előzményrekord tartalmazza a ticket azonosítóját, az előző és az új állapotot, a változás időpontját, valamint a változást kiváltó felhasználót, ha van ilyen. A ticket létrehozásakor az első állapotbejegyzésnél az előző állapot üres. Automatikus állapotváltás esetén a módosítást kiváltó felhasználó azonosítója lehet üres; a változás ettől függetlenül rögzül az előzményekben.

5.2.5 Bejegyzések, újranyitások és értesítések

A `TicketNotes` tábla tárolja a tickethez kapcsolódó kommunikációs bejegyzéseket. Egy bejegyzés egy tickethez és egy szerző felhasználóhoz tartozik. A bejegyzésekben a kommunikáció mellett az elvégzett munka leírása is rögzíthető; külön javítási tábla ezért nem szükséges.

A `TicketReopenings` tábla az újranyitás indoklását és időpontját őrzi meg. Egy ticket többször is újranyitható, ezért egy bejelentéshez több újranyitási rekord tartozhat.

A `Notifications` tábla tárolja az értesítés címzettjét, típusát, üzenetét, létrehozásának idejét és olvasottságát. Az értesítéshez ticket is kapcsolható; a `TicketId` üres lehet például olyan értesítésnél, amely nem egy konkrét bejelentéshez kapcsolódik.

5.2.6 A fő táblák és kapcsolataik összefoglalása

- Egy csoporthoz több felhasználó tartozhat; egy felhasználó legfeljebb egy csoporthoz rendelhető.
- Egy felhasználó több bejelentést hozhat létre.
- Egy felhasználó több ticket aktuális kezelője lehet.
- Egy csoport több bejelentést kaphat.
- Egy kategóriához több bejelentés tartozhat.
- Egy tickethez több kommunikációs bejegyzés, újranyitási rekord és állapotváltozás tar-

tozhat.

– Egy felhasználó több bejegyzést írhat, újranyitást kezdeményezhet és értesítést kaphat.

A kapcsolatok alapján az adatbázis megőrzi, ki hozta létre és ki kezelte a bejelentést, melyik csoport felelős érte, milyen kommunikáció történt, és hogyan változott az állapota.

5.3 Asztali alkalmazás

Az asztali alkalmazás a felhasználói szerepköröknek megfelelő funkciókat jelenít meg. A bejelentői, karbantartói és adminisztrátori felületek eltérő műveleteket tesznek elérhetővé, miközben ugyanahhoz a Web API-hoz kapcsolódnak.

A jelenlegi prototípus egy állapotoldalt tartalmaz, amely megjeleníti a backend és az adatbázis elérhetőségét, valamint lehetőséget ad az ellenőrzés megismétlésére. A lent leírt szerepkörspecifikus képernyők a rendszer tervezett funkciói; ezek teljes megvalósítása még hátravan.

5.3.1 Hibabejelentő GUI

A hibabejelentő felület célja, hogy a felhasználó gyorsan és érthetően rögzíthesse a tapasztalt problémát. A tervezett bejelentési folyamatban a felhasználó megadja a bejelentés címét, leírását, jellegét, kötelező helyszínét és – ha releváns – az érintett eszköz azonosítóját. A rendszer a bejelentés létrehozásakor egyedi azonosítót rendel hozzá.

A felhasználó megtekintheti saját bejelentéseinek listáját, megnyithatja azok részleteit, és követheti az aktuális állapotot. A bejelentéshez kapcsolódó kommunikációs előzményben új bejegyzést írhat. Lezárt bejelentés újranyitásakor indoklást kell megadnia.

5.3.2 IT Support GUI

A karbantartói felület a csoporthoz rendelt bejelentések áttekintését és kezelését támogatja. A karbantartó megnyithat egy bejelentést, részleteket olvashat róla, majd a csoport tagjaként magára veheti valamelyik feladatot.

A kezelés során a karbantartó bejegyzéseket fűzhet a bejelentéshez, rögzítheti az elvégzett munka leírását, és módosíthatja az állapotot. A feladat lezárásakor meg kell adni a lezárás módját, például azt, hogy a hiba elhárult, a bejelentést visszavonták, elutasították, vagy átmeneti megoldás született.

5.3.3 Rendszeradminisztrátor GUI

Az adminisztrátori felület a felhasználók, csoportok, kategóriák és bejelentések felügyeletét támogatja. A tervezett funkciók közé tartozik a regisztrációk jóváhagyása, a felhasználók inaktiválása, csoporttagságok kezelése, valamint a bejelentések karbantartói csoporthoz rendelése.

Az adminisztrátor a rendszer működéséhez szükséges kategóriákat és karbantartói csoportokat kezeli. A prioritások rögzített, kódban definiált értékek; ezek alkalmazásbeli módosítása nem adminisztrátori funkció.

6 Kommunikációs rendszer kialakítása (API)

6.1 Az API szerepe

Az alkalmazás kliens- és szerveroldali része HTTP-alapú API-n keresztül kommunikál. Az API fogadja a kliens kéréseit, ellenőrzi az adatokat és a jogosultságokat, meghívja a szükséges üzleti logikát, majd szabványos formátumú választ küld vissza. A kommunikáció tervezett adatformátuma a JSON.

A kliens nem kapcsolódik közvetlenül az adatbázishoz. A bejelentések lekérése, létrehozása vagy módosítása a Web API-n keresztül történik. Ez lehetővé teszi, hogy az üzleti szabályok ellenőrzése központilag, a szerveren valósuljon meg.

A jelenlegi prototípus már tartalmaz egy állapotellenőrző végpontot. A `GET /api/health` kérés azt ellenőrzi, hogy az API elérhető-e, és az adatbázissal létrehozható-e kapcsolat. A hibabejelentésekhez és felhasználói műveletekhez tartozó végpontok a további fejlesztés során valósítandók meg.

6.2 Tervezett végpontok

Az alábbi útvonalak példaként mutatják be a tervezett API-műveleteket. Nem teljes végpontjegyzék, és nem jelenti azt, hogy mindegyik szerepel a jelenlegi prototípusban. A végleges útvonalakat a megvalósított API alapján kell dokumentálni.

- `POST /api/auth/register` – új felhasználói regisztráció kezdeményezése.
- `POST /api/auth/login` – jóváhagyott felhasználó bejelentkezése.
- `GET /api/tickets` – a bejelentkezett felhasználó jogosultságának megfelelő bejelentések lekérése.
- `POST /api/tickets` – új hibabejelentés létrehozása.
- `GET /api/tickets/{id}` – egy bejelentés részleteinek lekérése.
- `GET /api/tickets/by-number/{ticketNumber}` – bejelentés keresése egyedi azonosító alapján.
- `POST /api/tickets/{id}/notes` – kommunikációs vagy munkavégzési bejegyzés hozzáadása.

– `POST /api/tickets/{id}/reopen` – lezárt bejelentés újranyitása indoklással.

– `POST /api/tickets/{id}/assign-group` – bejelentés karbantartói csoporthoz rendelése.

– `POST /api/tickets/{id}/take` – csoporthoz rendelt bejelentés átvétele.

– `POST /api/tickets/{id}/close` – bejelentés lezárása a lezárás módjának megadásával.

– `GET /api/notifications` – a bejelentkezett felhasználó értesítéseinek lekérése.

– `PATCH /api/notifications/{id}/read` – értesítés olvasottként jelölése.

– `GET /api/admin/registrations` – jóváhagyásra váró regisztrációk megtekintése.

– `POST /api/admin/registrations/{id}/approve` – regisztráció jóváhagyása.

6.3 Kérések és válaszok

A kliens a végpontoknak JSON-formátumú adatokat küldhet. Egy új bejelentés létrehozásakor a kérés törzse például a címet, a leírást, a hiba jellegét, a helyszínt, az opcionális eszközazonosítót és a kategória azonosítóját tartalmazhatja. A szerver ellenőrzi, hogy a szükséges adatok meg vannak-e adva, és a kapott azonosítók létező kategóriára hivatkoznak-e.

Sikeres létrehozás esetén a válasz tartalmazhatja a létrejött bejelentés belső azonosítóját, egyedi ticketazonosítóját, kezdeti állapotát és létrehozási idejét. A kliens ezt az információt felhasználva megnyithatja a bejelentés részleteit.

A válaszoknál célszerű a HTTP-státuszkódokat következetesen használni. Sikeres lekérés esetén például `200 OK`, sikeres létrehozás esetén `201 Created`, hibás bemenet esetén `400 Bad Request`, hiányzó vagy érvénytelen hitelesítésnél `401 Unauthorized`, elégtelen jogosultságnál `403 Forbidden`, nem létező erőforrásnál pedig `404 Not Found` adható vissza.

A hibaválaszoknak a kliens számára feldolgozható, egyértelmű hibaüzenetet kell tartalmazniuk. A belső adatbázis-lekérdezéseket, jelszavakat vagy részletes technikai kivételinformációkat nem szabad a felhasználónak megjelenített válaszba tenni.

6.4 Hitelesítés és jogosultság-ellenőrzés

A tervezett rendszerben a felhasználó regisztrációja adminisztrátori jóváhagyáshoz kötött. A még nem jóváhagyott vagy inaktivált felhasználó nem használhatja a bejelentkezést igénylő funkciókat. Az API-nak minden védett műveletnél ellenőriznie kell a kéréshez tartozó felhasználói azonosítót és a felhasználó jogosultságát.

A hozzáférés szabályait a szerepkör és a felhasználó adatai együtt határozzák meg. A bejelentő a saját bejelentéseit tekintheti meg. A karbantartó a saját bejelentései mellett a csoportjához rendelt bejelentéseket is elérheti. Az adminisztrátor a rendszer felügyeletéhez szükséges adatokhoz fér hozzá, és adminisztratív műveleteket végezhet.

A felhasználói felület szerepkörönként elrejthet vagy megjeleníthet műveleteket, de ez önmagában nem biztonsági ellenőrzés. A szervernek kell eldöntenie, hogy a kérő felhasználó ténylegesen végrehajthatja-e a műveletet. Ezzel elkerülhető, hogy valaki a kliens közvetlen módosításával olyan adatot érjen el, amelyre nincs jogosultsága.

A jelszavakat nem szabad olvasható formában tárolni. A megvalósítás során jelszóhash-elést kell használni, a hitelesítési adatok kezelését pedig a Web API feladatkörében kell megoldani. A konkrét hitelesítési megoldás és a munkamenet lejárati szabályai a fejlesztési terv további részletei.

6.5 A bejelentés életciklusa

A bejelentés létrehozásakor a rendszer egyedi azonosítót rendel hozzá, rögzíti a bejelentőt és a létrehozás idejét, valamint kezdeti állapotot és alapértelmezett prioritást állít be. A határidőt a kiválasztott prioritáshoz rendelt, kódban konfigurált munkanapszám alapján kell kiszámítani.

A bejelentést az adminisztrátor vagy az erre jogosult felhasználó karbantartói csoporthoz rendelheti. A csoport tagjai látják a hozzájuk rendelt feladatot; egyikük átveheti, és ettől kezdve ő jelenik meg a bejelentés aktuális kezelőjeként.

A kezelés során a karbantartó bejegyzéseket fűzhet a bejelentéshez. Ha a karbantartó választ vagy kérdést küld a bejelentőnek, a bejelentés a terv szerint Válaszra vár állapotba kerül. Amikor a bejelentő válaszol, a bejelentés ismét Folyamatban állapotú lesz.

A karbantartó a feladat elvégzése után lezárja a bejelentést, és megadja a lezárás módját. A bejelentő a lezárást követően is megtekintheti a saját bejelentésének előzményeit. Ha a

probléma nem szűnt meg, indoklással újranyithatja a bejelentést.

Az állapotváltozások minden esetben bekerülnek a státuszelőzmény-táblába. Így az adatbázis az aktuális állapot mellett azt is megőrzi, hogy a bejelentés korábban milyen állapotokon ment keresztül.

6.6 Értesítések kezelése

Az értesítések célja, hogy a felhasználók észrevegyék a számukra fontos változásokat. A tervezett rendszer alkalmazáson belüli értesítéseket használ; e-mailes értesítés jelenleg nem része a követelményeknek.

A bejelentő értesítést kaphat a saját bejelentésének állapotváltozásáról, új bejegyzésről és újranyitásról. A karbantartó értesítést kaphat a csoportjához rendelt új bejelentésről és a bejelentő válaszáról. Az adminisztrátor értesítést kaphat új regisztrációról.

Az értesítéshez legalább a címzett felhasználót, az esemény típusát, a kapcsolódó bejelentést, az üzenetet, a létrehozás idejét és az olvasottság állapotát kell tárolni. A konkrét értesítési szövegek és az olvasottként jelölés működése a kliens és az API megvalósításakor pontosítható.

7 Megvalósítás menete és tesztelés

7.1 A jelenlegi prototípus állapota

A projekt jelenlegi állapotában elkészült a .NET Web API és a .NET MAUI kliens alapváza. A Web API SQLite-adatbázishoz kapcsolódik, és rendelkezik egy állapotellenőrző végponttal. A végpont ellenőrzi az adatbázis-kapcsolatot, és a szolgáltatás állapotáról választ ad.

A MAUI-kliens az API állapotát HTTP-kérés segítségével lekéri, majd megjeleníti, hogy a backend és az adatbázis elérhető-e. A felületen az ellenőrzés ismételt indítására is van lehetőség.

A jelenlegi prototípus még nem tartalmazza a teljes felhasználói folyamatot. A bejelentkezés, a regisztráció-jóváhagyás, a ticketek kezelése, a csoportos kiosztás, az értesítések és a szerepkörök szerinti jogosultság-ellenőrzés további fejlesztést igényel. Ezeket a funkciókat a dokumentáció tervezett működésként mutatja be.

7.2 Javasolt fejlesztési sorrend

A fejlesztés egymásra épülő szakaszokban végezhető el. Elsőként az adatbázis entitásait és kapcsolatait kell véglegesíteni, majd a szerveroldali adatkezelési alapokat kialakítani. Ezután érdemes megvalósítani a felhasználói azonosítást és a jogosultságok ellenőrzését.

A bejelentés létrehozása, listázása és részleteinek megtekintése adja az első teljes felhasználói folyamatot. Ezt követheti a karbantartói csoporthoz rendelés, az átvétel, a bejegyzések kezelése, majd a lezárás és újranyitás. Az értesítések és az adminisztrátori funkciók a fő folyamatok stabil működése után építhetők be.

A kliensoldali képernyők fejlesztése során minden funkciót össze kell kapcsolni a megfelelő API-végponttal. A képernyőknek kezelniük kell a betöltési állapotot, az üres eredményt, a sikertelen kérést és a szerver által visszaküldött validációs hibát is.

7.3 Tesztelési terv

A tesztelés célja annak ellenőrzése, hogy az egyes funkciók a meghatározott követelmények szerint működnek, és hogy a jogosulatlan vagy hibás műveleteket a rendszer megfelelően kezeli. A teszteseteket szerepkörönként és felhasználói folyamatonként célszerű összeállítani.

A tervezett tesztelési területek a következők:

- **API elérhetősége:** az állapotellenőrző végpont elérhető, és helyesen jelzi az adatbázis-kapcsolat eredményét.
- **Regisztráció és jóváhagyás:** a jóváhagyásra váró felhasználó nem fér hozzá védett funkciókhoz, a jóváhagyott felhasználó pedig be tud jelentkezni.
- **Jogosultságok:** a bejelentő nem tekintheti meg más felhasználó privát bejelentéseit; a karbantartó csak a saját csoportjához tartozó feladatokat kezelheti; adminisztratív műveletet csak adminisztrátor végezhet.
- **Bejelentés létrehozása:** a kötelező mezők ellenőrzése, az azonosító létrehozása és az alapértelmezett értékek beállítása.
- **Állapotváltások:** csak a megengedett műveletek hajthatók végre, a lezárás és újranyitás pedig megőrzi a korábbi adatokat és a státuszelőzményeket.
- **Kommunikációs bejegyzések:** a bejegyzés a megfelelő felhasználóhoz és bejelentéshez kapcsolódik, az esemény pedig a meghatározott állapotváltozást eredményezi.
- **Értesítések:** az előírt események értesítést hoznak létre a megfelelő címzett számára.
- **Határidő számítása:** a rendszer a prioritáshoz rendelt munkanapszám alapján számítja ki a határidőt.
- **Hibás bemenet:** üres vagy hibás adatok esetén a rendszer visszautasítja a műveletet, és érthető hibaüzenetet ad.

A tesztelés során rögzíteni kell a teszt lépéseit, a várt eredményt és a tényleges eredményt. A dokumentációban csak ténylegesen lefuttatott tesztek eredménye szerepelhet kész eredményként; a még le nem futtatott eseteket tesztelési tervként kell megjelölni.

7.4 Ismert korlátok és továbbfejlesztési lehetőségek

A prototípus jelenlegi korlátja, hogy még csak az API és az adatbázis kapcsolatának ellenőrzését mutatja be. A teljes bejelentési folyamat és a hozzá tartozó szerepkörök megvalósítása még szükséges. A felhasználói felület egyelőre nem tartalmazza a végleges bejelentői, karbantartói és adminisztrátori nézeteket.

A további fejlesztés során megvalósítható a részletesebb keresés, például állapot, kategória,

prioritás és dátum alapján. Későbbi bővítés lehet a csatolmányok kezelése, részletesebb statisztikák vagy külső értesítési csatorna bevezetése. Ezek nem részei az első változat minimális funkcióinak, ezért csak a fő folyamatok elkészülte után érdemes velük foglalkozni.

8 Összegzés

A projekt célja egy olyan asztali hibabejelentő és karbantartás-kezelő rendszer megtervezése és megvalósítása, amelyben a felhasználók rögzíthetik a problémákat, a karbantartók kezelhetik azokat, az adminisztrátor pedig felügyelheti a rendszer működését. A csapat a kiírás általános műszaki hibakezelési feladatát vállalati IT-infrastruktúrára szűkíti.

A rendszer tervezett architektúrája .NET MAUI kliensből, .NET Web API-ból és szerveroldali SQLite-adatbázisból áll. A prioritások kódban definiált enumértékek, a határidőket a rendszer munkanapok alapján számítja. A bejelentések állapotváltozásai külön előzménytáblában rögzülnek.

A jelenlegi prototípusban már működik az API és az adatbázis kapcsolatának ellenőrzésére szolgáló alap, de a teljes felhasználói és ticketkezelési folyamat további fejlesztést igényel. A következő fejlesztési szakasz feladata az adatmodell véglegesítése, majd a hitelesítés és az alapvető bejelentéskezelés megvalósítása. A dokumentáció a projekt előrehaladtával frissítendő, hogy a tervezett megoldások és a ténylegesen elkészült funkciók egyértelműen elkülönüljenek.

Irodalomjegyzék